



Universidad Autónoma de
San Luis Potosí

FACULTAD DE INGENIERÍA



Manual del programador CDiccionario

CARRERA:

Ingeniería en Sistemas Inteligentes

ASIGNATURA:

Lenguajes de programación

ELABORADO POR:

369664 Antonio Jiménez José Emilio
388531 Alvarez Puente Alan Antonio

DOCENTE:

JUAN JOSE ACOSTA ESPINOZA

CICLO ESCOLAR:

2025 - 2026

San Luis Potosí, S.L.P.

25 de mayo de 2026

Índice

1. Estructuras de Datos y Objetos	4
Entidad	4
Atributos	5
CDiccionario	6
2. Funciones Entidad	9
Menu	9
Nuevo	9
Abrir	9
3. Funciones Entidad	10
Menu Entidades	10
Captura Entidad	10
Alta Entidad	10
Busca Entidad	10
Get Cabecera Entidades	11
Lee Entidad	11
Escribe Cabecera Entidades	11
Escribe Entidad	11
Inserta Entidad	11
Diagrama Insercción en Entidad	12
Consulta Entidades	12
Baja Entidad	12
Elimina Entidad	12
Diagrama Eliminación en Entidad	12
Reescribe Entidad	12
Modifica Entidad	13
4. Funciones Atributo	14
Menu Atributos	14
Captura Atributo	14
Alta Atributo	14
Buscar Atributo	14
Lee Atributo	14
Escribe Atributo	14
Inserta Atributo	15
Diagrama Insercción en Atributo	15
Consultar Atributo	15
Baja Atributo	15
Elimina Atributo	16
Diagrama Eliminación en Atributo	16
Reescribe Atributo	16

Modifica Atributo	16
Carga Atributos	16
5. Funciones Bloque	18
Menu Datos	18
Captura Bloque	18
Compara Bloques	18
Alta Bloque	18
Reescribe Bloque	18
Lee Bloque	18
Escribe Bloque	19
Inserta Bloque	19
Diagrama Insercción en Bloque	19
Consulta Bloques	19
Busca Bloque	20
Modifica Bloque	20
Baja Bloque	20
Diagrama Eliminación en Bloque	20
Pide Clave Bloque	20
Elimina Bloque	21
6. Diagrama General	22
Arquitectura del Diccionario	22
7. Archivo	23
Cabecera Entidad	23
Eliminación Lógica (Baja)	23
Funciones de Archivo Binario	23
8. Simulaciones de Escritorio: Entidad	24
1. Simulación: CapturaEntidad	24
2. Simulación: escribeEntidad	24
3. Simulación: insertaEntidad	24
4. Simulación: ConsultaEntidades	25
9. Simulaciones de Escritorio: Atributos	26
1. Simulación: CapturaAt	26
2. Simulación: EscribeAtributo	26
3. Simulación: insertaAtributo	26
4. Simulación: consultarAtributo	26
10. Simulaciones de Escritorio: Bloques	28
1. Simulación: CapturaBloque	28
2. Simulación: escribeBloque	28
3. Simulación: InsertaBloque	28
4. Simulación: consultaBloques	29

11. Clave Primaria	30
¿Qué es una Clave Primaria?	30
Manejo de la Clave Primaria en el Sistema	30
12. Conclusiones	31

1 Estructuras de Datos y Objetos

Descripción de la Estructura Entidad:

La estructura es el molde inicial de nuestro Diccionario en formato de base de datos de acceso aleatorio. Se establece un alias `cadena` para asegurar que cada registro tenga el mismo tamaño.

- **cadena nombre:** Identificador único de cada registro de entidad (máximo 30 caracteres).
- **long atr:** Puntero tipo `long` que conecta la entidad con su lista de atributos.
- **long data:** Dirección física en el archivo donde se almacenan los registros de datos reales.
- **long sig:** Enlace a la siguiente entidad, permitiendo una estructura de lista ligada.

```
1 #ifndef ENTIDAD_H_INCLUDED
2 #define ENTIDAD_H_INCLUDED
3 #include <iostream>
4 #include <cstdio>
5
6 typedef char cadena[30];
7 struct Entidad
8 {
9     cadena nombre;
10    long atr;
11    long data;
12    long sig;
13 };
14 #endif
```

Estructura 1: Estructura de Datos de Entidad

Descripción de la Estructura Atributo:

Esta estructura es el molde para las columnas de nuestro Diccionario. Al igual que en la entidad, se usa el alias `cadena` para mantener el tamaño fijo en el archivo binario y evitar que los registros se desfasen. En esta estructura observamos las siguientes declaraciones:

- **cadena nombre:** Identificador del atributo dentro de la entidad.
- **unsigned int tipo:** Código numérico que define el tipo de dato (ej. entero, flotante, cadena).
- **unsigned int tamano:** Longitud en bytes que ocupará el valor del atributo en el registro.
- **char iskp:** Bandera (flag) que indica si el atributo funciona como llave primaria (Primary Key).
- **long sig:** Puntero lógico (offset) que enlaza con el siguiente atributo de la entidad.
- **cadena descripcion:** Información adicional o etiqueta descriptiva del campo.
- **char null:** Indicador booleano que define si el campo permite valores nulos.

```
1 #ifndef ATRIBUTO_H
2 #define ATRIBUTO_H
3
4 typedef char cadena [30];
5
6 typedef struct{
7     cadena nombre;
8     unsigned int tipo;
9     unsigned int tamano;
10    char iskp;
11    long sig;
12    cadena descripcion;
13    char null;
14 }Atributo;
15
16 #endif
```

Estructura 2: Estructura de Datos de Atributos

Descripción del Objeto CDiccionario:

La clase CDiccionario funciona como el administrador principal del sistema. En su parte privada, contiene las variables necesarias para controlar el flujo del archivo binario y saber exactamente en qué parte del disco estamos parados mientras el usuario navega. En esta sección observamos las siguientes declaraciones:

- **FILE *arch:** Es el puntero al archivo físico donde se almacenan todas las entidades, atributos y datos.
- **Entidad activa:** Representa la entidad que el usuario está manipulando o consultando en ese momento.
- **long dir:** Guarda la dirección física (offset) actual dentro del archivo para las operaciones de lectura/escritura.
- **int nAtributos:** Es un contador que lleva el registro de cuántos atributos tiene la entidad seleccionada.
- **long tambloque:** Almacena el tamaño del bloque activo para gestionar correctamente el espacio en el archivo.
- **char nombreArchivo[50]:** Es el arreglo donde se guarda el nombre del archivo que el usuario decide abrir o crear.

```
1  #ifndef CDICCIONARIO_H_INCLUDED
2  #define CDICCIONARIO_H_INCLUDED
3
4  #include <cstdio>
5  #include "Atributo.h"
6  #include "Entidad.h"
7
8  class CDiccionario
9  {
10 private:
11     FILE *arch;
12     Entidad activa;
13     long dir;
14     int nAtributos;
15     long tambloque;
16     Atributo arrAtributos[50];
17     char nombreArchivo[50];
18 public:
19     //constructor
20     CDiccionario();
21
22     //Entidades
23     void Menu_Entidades();
24     Entidad CapturaEntidad();
25     void altaEntidad();
26     long buscaEntidad(char nombre[30]);
27     long getcabeceraEntidades();
```

```
28  Entidad leeEntidad(long dir);
29  void EscribeCabEntidades(long cab);
30  long escribeEntidad(Entidad ant);
31  void insertaEntidad(Entidad nvo, long dir);
32  void ConsultaEntidades();
33  void BajaEntidad();
34  long EliminaEntidad(char nombre[30]);
35  void reescribeEntidad(long dir, Entidad ant);
36  void modificaEntidad();
37  void preguntaEntidad();
38
39
40  //Atributos
41  void Menu_Atributos();
42  void consultarAtributo();
43  long eliminaAtributo(char cad[30]);
44  void modificaAtributo();
45  void AltaAtributo();
46  Atributo CapturaAt();
47  void insertaAtributo(Atributo nvo, long dir);
48  long BuscarAtributo(char *atr);
49  Atributo leeAtributo(long dir);
50  long EscribeAtributo(Atributo ant);
51  void reescribeAtributo(long dir, Atributo atr);
52  void BajaAtributo();
53  int cargaAtributos();
54
55
56  //Datos
57  void Menu_Datos();
58  void* CapturaBloque();
59  float comparaBloques(void*b1, void*b2);
60  void altaBloque();
61  void reescribeBloque(void* bloque, long dir);
62  void* LeeBloque(long dir);
63  long escribeBloque(void* bloque);
64  void InsertaBloque(void *nvo, long dirnvo);
65  void consultaBloques();
66  long buscaBloque(void *bloquebus);
67  void modificaBloque();
68  void bajaBloque();
69  void eliminaBloque(void*bloque);
70  void *pideClaveBloque();
71
72
73
74
75
```

```
76     //Principal
77     void menu();
78     void Nuevo();
79     void abrir();
80
81 };
82
83 #endif // CDICCIONARIO_H_INC
```

Estructura 3: Estructura de Datos CDiccionario

2 Funciones

Archivo

Menu

Prototipo: `void CDiccionario::menu();`

La función `menu()` simplemente muestra en consola las opciones principales del sistema, imprimiendo las opciones para crear un archivo nuevo, abrir un archivo existente o salir del programa, sirviendo como interfaz inicial para el usuario.

Nuevo

Prototipo: `void CDiccionario::Nuevo();`

Esta función es la encargada de crear un archivo binario desde cero, primero pide al usuario el nombre del archivo, después intenta abrirlo con `rb` para verificar que no exista previamente y evitar sobreescribirlo, en caso de que no exista lo crea con `wb+`, inicializa la cabecera de entidades escribiendo un `-1` al inicio del archivo, y finalmente redirige al usuario llamando a `Menu_Entidades()`.

Abrir

Prototipo: `void CDiccionario::abrir();`

Esta función permite cargar un diccionario de datos previamente creado, solicita al usuario el nombre del archivo y hace uso de `fopen` con la instrucción `rb+` para lectura y escritura, verifica si el archivo es nulo para avisar que no existe, en caso contrario confirma su apertura y manda a llamar a la función `Menu_Entidades()` para comenzar a trabajar.

3 Funciones

Entidad

Menu Entidades

Prototipo: `void CDiccionario::Menu_Entidades();`

La función `menu_Entidades()` muestra un menú interactivo por consola para administrar las funciones relacionadas a Entidades. Funciona por medio de un ciclo `do-while` y un `switch-case` que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar entidades, por consiguiente permitir la navegación hacia la edición de sus atributos y datos.

Captura Entidad

Prototipo: `Entidad CDiccionario::CapturaEntidad();`

La función `CapturaEntidad()` es la plantilla principal para crear registros, se usa una variable de tipo `Entidad`, primero se le pide al usuario el nombre de la entidad, después se inicializa las variables `nueva.atr` "conexión con la lista de atributos", `nueva.data` "la dirección" real de donde se encuentran los datos de los bloques, y `nueva.sig` que es la dirección a la siguiente entidad, con esto se crea un registro nuevo con sus listas vacías.

Alta Entidad

Prototipo: `void CDiccionario::altaEntidad();`

Esta función es la encargada de verificar que todo esté en condiciones para que se cree una entidad, primero manda a llamar `CapturaEntidad` para crear un registro temporal, seguido de esto viene la verificación con la función `buscaEntidad` para verificar que no haya una entidad con ese mismo nombre, en caso que no exista usa la función `escribeEntidad(nueva)` y guarda la dirección en un `long dir` para finalmente usar la función `insertaEntidad` para escribir en el archivo.

Busca Entidad

Prototipo: `long CDiccionario::buscaEntidad(char nombre[1]);`

La función `long CDiccionario::buscaEntidad(char nombre[])` verifica si se encuentra la entidad, en caso de que sí la encuentre dentro del recorrido `while(cab != -1)` regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, dentro del ciclo hace un string compare de la posición donde se encuentra y el nombre que se le haya pasado.

Get Cabecera Entidades

Prototipo: `long CDiccionario::getcabeceraEntidades();`

Esta función es la encargada de obtener la dirección dentro del archivo de donde inicia nuestra lista de entidades, empieza usando un `long dir`, para después con `fseek` dirigirse al inicio de nuestro archivo, con un `fread` extraemos la dirección y la guardamos en la variable `long dir`, sabemos que esa es la cabecera ya que todo el programa está construido para que en el inicio del archivo se encuentra nuestra cabecera de la lista de entidades.

Lee Entidad

Prototipo: `Entidad CDiccionario::leeEntidad(long dir);`

Se encarga de extraer la entidad del archivo, esta función llega con una dirección, esa dirección a partir del `buscaEntidad`, es usada para moverse dentro del archivo, hacer una copia y regresar esa copia para su posterior modificación o consulta.

Escribe Cabecera Entidades

Prototipo: `void CDiccionario::EscribeCabEntidades(long cab);`

La función empieza con un paso de parámetro de la cabecera, dentro de esta se mueve al inicio del archivo y en ese inicio escribe lo que haya tenido el parámetro de nombre `cab`.

Escribe Entidad

Prototipo: `long CDiccionario::escribeEntidad(Entidad ant);`

Esta función se encarga de registrar físicamente una nueva entidad en el archivo, donde se le pasa una copia de la entidad, la función usa un `fseek` con `SEEK_END` para ir hasta el final del último registro del archivo y en ese espacio de memoria usa un `fwrite` para guardarlo en el archivo.

Inserta Entidad

Prototipo: `void CDiccionario::insertaEntidad(Entidad nvo, long dir);`

Esta función se encarga de enlazar secuencialmente de manera lógica una nueva entidad verificando tres casos: si el archivo está vacío, si va al inicio apoyándose de un `strcmp`, o si busca su lugar usando un ciclo `while`. En esencia, como `escribeEntidad` siempre guarda físicamente los datos al final, esta función solo reconecta las variables `nvo.sig` para que apunten de manera correcta y mantengan el orden.

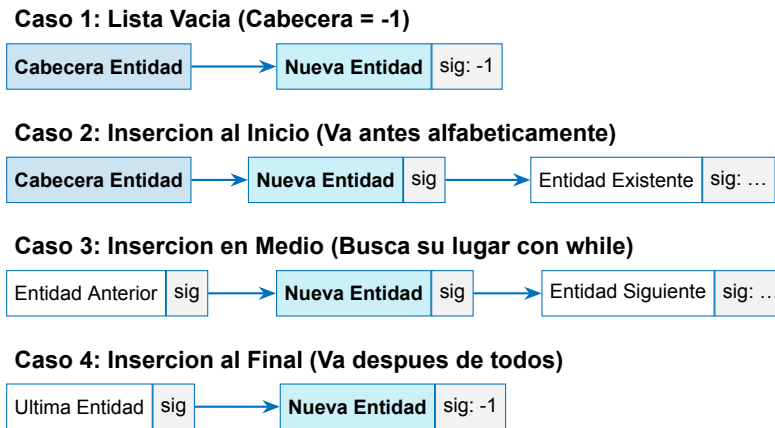


Diagrama 1: Representacion grafica de los 4 casos de insercion logica para la lista de Entidades.

Consulta Entidades

Prototipo: `void CDiccionario::ConsultaEntidades();`

Esta función es la encargada de imprimir en consola las entidades que se encuentran dentro de nuestro archivo. Para ello, hace uso de la función `getcabeceraEntidades` para ver dónde inicia la cabecera de la entidad; subsecuentemente hace uso de un ciclo `while (cab != -1)`, el cual va imprimiendo cada entidad con la dirección de sus componentes.

Baja Entidad

Prototipo: `void CDiccionario::BajaEntidad();` Su unica funcion es preguntar al usuario por el nombre de la entidad, este nombre se lo pasa a la función `EliminaEntidad` en caso de que no exista muestra en consola un aviso al usuario, por el contrario si existe, muestra un aviso que fue eliminado.

Elimina Entidad

Prototipo: `long CDiccionario::EliminaEntidad(char nombre[]);` Esta funcion empieza haciendo una verificación, si el archivo esta vacio regresa un -1, para despues verificar si el lugar actual donde esta parado es la entidad a borrar, en caso que no hace un recorrido donde hace uso de `while & strcmp`, para encontrar donde esta, uso una variable anterior para poder hacer la desconexion de la entidad, no borra nada dentro del archivo solo desceonecta y conecta con su inmediato siguiente, lo actualiza en el archivo con la función `reescribeEntidad`.

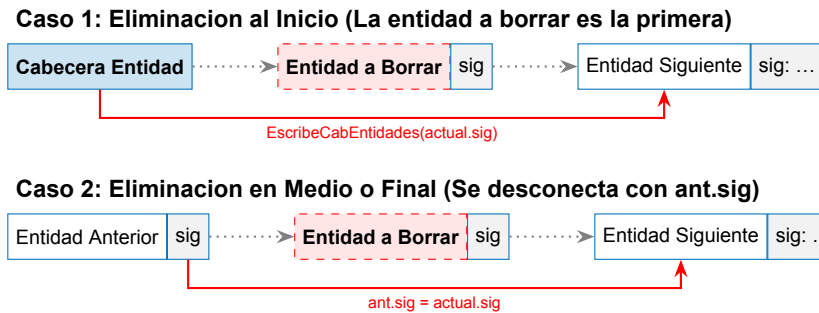


Diagrama 2: Representacion grafica de los 2 casos de baja logica para la lista de Entidades. El registro no se borra fisicamente, solo se puentea.

Reescribe Entidad

Prototipo: `void CDiccionario::reescribeEntidad(long dir, Entidad ant);`

Esta función se encarga de actualizar los datos de un registro ya existente en el archivo binario. Recibe una dirección y una estructura con la información modificada, usa `fseek` con `SEEK_SET` para posicionarse exactamente en la dirección proporcionada, y `fwrite` para sobrescribir el bloque de datos. Es indispensable para actualizar los apuntadores lógicos durante operaciones de inserción y baja.

Modifica Entidad

Prototipo: `void CDiccionario::modificaEntidad();`

Esta función permite actualizar los datos de un registro. Tras verificar que existencia y capturar la nueva información, hace uso de una estrategia segura para mantener el orden alfabético de la lista: elimina lógicamente el registro antiguo con `EliminaEntidad` y despues guarda y reubica la entidad actualizada utilizando `escribeEntidad` e `insertaEntidad`.

4 Funciones

Atributo

Menu Atributos

Prototipo: void CDiccionario::Menu_Atributos();

La función Menu_Atributos() muestra un menú interactivo por consola para administrar las funciones relacionadas a Atributos. Funciona por medio de un ciclo do-while y un switch-case que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar atributos, por consiguiente permitir la navegación hacia la edición de sus componentes.

Captura Atributo

Prototipo: Atributo CDiccionario::CapturaAt();

La función CapturaAt() es la plantilla principal para crear registros, se usa una variable de tipo Atributo, primero se le pide al usuario el nombre, después se pregunta el tipo y tamaño, y se inicializan las variables nuevoA.iskp y nuevoA.null, y nuevoA.sig en -1, con esto se crea un registro nuevo con sus datos listos.

Alta Atributo

Prototipo: void CDiccionario::AltaAtributo();

Esta función es la encargada de verificar que todo esté en condiciones para que se cree un atributo, primero manda a llamar CapturaAt para crear un registro temporal, seguido de esto viene la verificación con la función BuscarAtributo para verificar que no haya un atributo con ese mismo nombre, en caso que no exista usa la función EscribeAtributo(nuevoA) y guarda la dirección en un long dir para finalmente usar la función insertaAtributo para escribir en el archivo.

Buscar Atributo

Prototipo: long CDiccionario::BuscarAtributo(char *atr);

La función long CDiccionario::BuscarAtributo(char *atr) verifica si se encuentra el atributo, en caso de que sí lo encuentre dentro del recorrido while(cab != -1) regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, dentro del ciclo hace un string compare de la posición donde se encuentra y el nombre que se le haya pasado.

Lee Atributo

Prototipo: Atributo CDiccionario::leeAtributo(long dir);

Se encarga de extraer el atributo del archivo, esta función llega con una dirección, esa dirección a partir del BuscarAtributo, es usada para moverse dentro del archivo, hacer una copia y regresar esa copia para su posterior modificación o consulta.

Escribe Atributo

Prototipo: `long CDiccionario::EscribeAtributo(Atributo ant);`

Esta función se encarga de registrar físicamente un nuevo atributo en el archivo, donde se le pasa una copia del atributo, la función usa un `fseek` con `SEEK_END` para ir hasta el final del último registro del archivo y en ese espacio de memoria usa un `fwrite` para guardarlo en el archivo.

Inserta Atributo

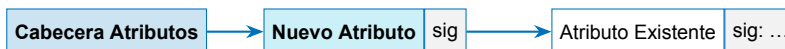
Prototipo: `void CDiccionario::insertaAtributo(Atributo nvo, long dir);`

Esta función se encarga de enlazar secuencialmente de manera lógica un nuevo atributo verificando tres casos: si en el caso que la cabecera `activa.atr` sea -1 "Lista vacía", se enlaza directo, el segundo caso ocurre cuando el atributo se encuentra al inicio, o sea que el `strcmp` sea mayor a 0 decidiendo así que va antes, el tercer caso es cuando tiene que hacer una búsqueda apoyándose de un `while` que recorre toda la lista mientras no llegue al final, dentro usa otro `strcmp` para ver en qué lugar se encuentra su inmediato.
 newline En esencia la función hace reconexión entre la variable `nvo.sig` para que quede de manera ordenada, todos los `EscribeAtributo` siempre hacen el `write` al final por tanto lo único que cambia esta función son las direcciones a las cuales van apuntando de manera ordenada.

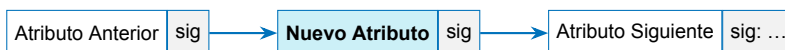
Caso 1: Lista Vacía (`activa.atr = -1`)



Caso 2: Insercion al Inicio (Va antes alfabeticamente)



Caso 3: Insercion en Medio (Busca su lugar con while)



Caso 4: Insercion al Final (Va despues de todos)



Diagrama 3: Representacion grafica de los 4 casos de insercion logica para la lista de Atributos.

Consultar Atributo

Prototipo: `void CDiccionario::consultarAtributo();`

Esta función es la encargada de imprimir en consola los atributos que se encuentran dentro de nuestro archivo. Para ello, hace uso de `activa.atr` para ver dónde inicia la cabecera del atributo; subsecuentemente hace uso de un ciclo `while` (`cab != -1`), el cual va imprimiendo cada atributo con la dirección de sus componentes.

Baja Atributo

Prototipo: `void CDiccionario::BajaAtributo();`

Su única función es preguntar al usuario por el nombre del atributo, este nombre se lo pasa a la función `eliminaAtributo` en caso de que no exista muestra en consola un aviso al usuario, por el contrario si existe, se hace la desconexión.

Elimina Atributo

Prototipo: `long CDiccionario::eliminaAtributo(char cad[]);`

Esta función empieza verificando la posición desde `activa.atr`, para después verificar si el lugar actual donde está parado es el atributo a borrar, en caso que no hace un recorrido donde hace uso de `while & strcmp`, para encontrar dónde está, usa una variable anterior para poder hacer la desconexión del atributo, no borra nada dentro del archivo solo desconecta y conecta con su inmediato siguiente, lo actualiza en el archivo con la función `reescribeAtributo`.

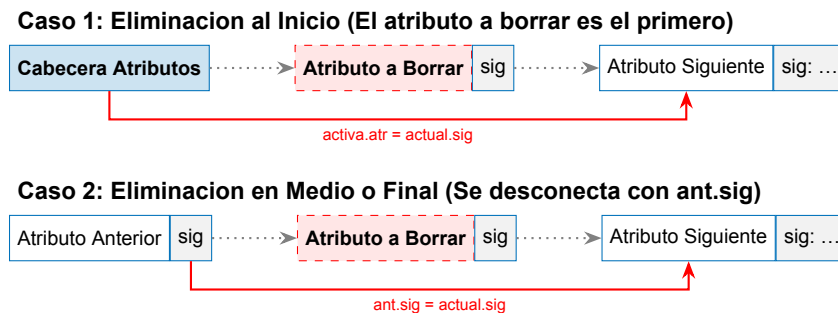


Diagrama 4: Representación gráfica de los 2 casos de baja lógica para la lista de Atributos.

Reescribe Atributo

Prototipo: `void CDiccionario::reescribeAtributo(long dir, Atributo atr);`

Esta función se encarga de actualizar los datos de un registro ya existente en el archivo binario. Recibe una dirección y una estructura con la información modificada, usa `fseek` con `SEEK_SET` para posicionarse exactamente en la dirección proporcionada, y `fwrite` para sobrescribir el bloque de datos. Es indispensable para actualizar los apuntadores lógicos durante operaciones de inserción y baja.

Modifica Atributo

Prototipo: `void CDiccionario::modificaAtributo();`

Esta función permite actualizar los datos de un registro. Tras verificar su existencia y capturar la nueva información, hace uso de una estrategia segura para mantener el orden alfabético de la lista: elimina lógicamente el registro antiguo con `eliminaAtributo` y después guarda y reubica el atributo actualizado utilizando `reescribeAtributo` e `insertaAtributo`.

Carga Atributos

Prototipo: `int CDiccionario::cargaAtributos();`

Esta función es la encargada de preparar todo para trabajar con los registros realizando tres tareas principales, primero recorre los atributos con un ciclo `while` para ir sumando su tamaño y definir el tamaño, después va guardando cada atributo dentro del arreglo `arrAtributos` e incrementa el contador `nAtributos`, y por último verifica si el atributo es clave primaria comprobando la variable `iskp` para intercambiarlo y ponerlo siempre en la posición 0 del arreglo, finalizando con el retorno del número de claves encontradas.

5 Funciones

Bloque

Menu Datos

Prototipo: `void CDiccionario::Menu_Datos();` La función `Menu_Datos()` muestra un menú interactivo por consola para administrar las funciones relacionadas a los Bloques o Registros. Funciona por medio de un ciclo `do-while` y un `switch-case` que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar registros, permitiendo la navegación hacia la edición de los datos.

Captura Bloque

Prototipo: `void* CDiccionario::CapturaBloque();`
La función `CapturaBloque()` es la plantilla principal para crear registros de datos, se usa un puntero `void* bloque` usando `malloc` con el tamaño `tambloque`, primero inicializa el apuntador al siguiente, después usa un ciclo `for` para pedirle al usuario los datos dependiendo del tipo de atributo, con esto se crea un bloque nuevo con su información lista.

Compara Bloques

Prototipo: `float CDiccionario::comparaBloques(void* b1, void *b2);`
Esta función es la encargada de comparar dos bloques apoyándose de la clave primaria que siempre está en el índice 0 del arreglo, primero verifica el tipo de dato, en el caso de ser una cadena hace uso de `strcmp` para compararlos, mientras que para los tipos numéricos enteros o flotantes realiza una resta directa de sus valores, regresando un resultado que sirve para saber si son iguales o decidir cuál va antes.

Alta Bloque

Prototipo: `void CDiccionario::altaBloque();`
Esta función es la encargada de verificar que todo esté en condiciones para crear un bloque, primero manda a llamar `CapturaBloque` para crear el registro, seguido de esto viene la verificación con la función `buscaBloque` para comprobar que no exista ya esa clave primaria, en caso que no exista usa `escribeBloque` obteniendo su dirección, y finalmente usa `InsertaBloque` para escribir en el archivo.

Reescribe Bloque

Prototipo: `void CDiccionario::reescribeBloque(void* bloque, long dir);`
Esta función se encarga de actualizar los datos de un registro ya existente en el archivo binario. Recibe una dirección y un bloque con la información, usa `fseek` con `SEEK_SET` para posicionarse exactamente en la dirección proporcionada, y `fwrite` para sobrescribir haciendo uso de la variable `tambloque`. Es indispensable para actualizar apuntadores.

Lee Bloque

Prototipo: void* CDiccionario::LeeBloque(long dir);

Se encarga de extraer el bloque del archivo, esta función llega con una dirección, la cual es usada por un `fseek` para moverse dentro del archivo y con un `fread` hacer una copia de tamaño `tambloque` en un espacio de memoria, regresando esa copia para su posterior modificación o consulta.

Escribe Bloque

Prototipo: long CDiccionario::escribeBloque(void* bloque);

Esta función se encarga de registrar físicamente un nuevo bloque en el archivo, donde se le pasa el puntero de los datos, la función usa un `fseek` con `SEEK_END` para ir hasta el final del último registro del archivo, guarda su dirección con `ftell` y en ese espacio usa un `fwrite` para guardarlo en el archivo, regresando la dirección física.

Inserta Bloque

Prototipo: void CDiccionario::InsertaBloque(void *nvo, long dirnvo);

Esta función se encarga de enlazar secuencialmente de manera lógica un nuevo bloque, si en el caso que la cabecera `activa.data` sea `-1` se enlaza directo, si va al inicio o en medio usa `comparaBloques` dentro de un `while` para hacer una búsqueda y encontrar su lugar exacto.

newline En esencia la función hace reconexión de las direcciones para que quede de manera ordenada, como `escribeBloque` siempre guarda al final, lo único que cambia esta función son los apuntadores lógicos.

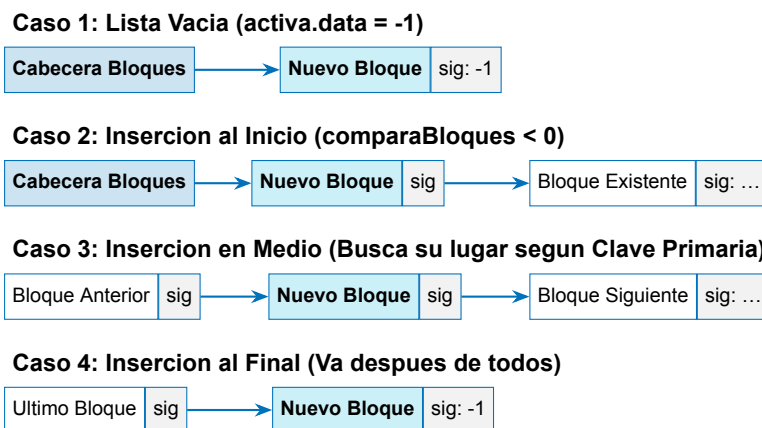


Diagrama 5: Representación gráfica de los 4 casos de inserción lógica para la lista de Bloques de Datos.

Consulta Bloques

Prototipo: void CDiccionario::consultaBloques();

Esta función es la encargada de imprimir en consola los registros que se encuentran dentro del archivo. Para ello, hace uso de `activa.data` para ver dónde inicia la cabecera, imprime los nombres de los atributos, y subsecuentemente hace uso de un ciclo `while (cab != -1)` el cual va imprimiendo cada bloque haciendo las conversiones de tipo correspondientes.

Busca Bloque

Prototipo: long CDiccionario::buscaBloque(void *bloquebus);

La función `buscaBloque` verifica si se encuentra el registro, en caso de que sí lo encuentre dentro del recorrido `while(cab != -1)` comparando con la función `comparaBloques` regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, moviéndose siempre leyendo el apuntador siguiente de cada bloque.

Modifica Bloque

Prototipo: void CDiccionario::modificaBloque();

Esta función permite actualizar los datos de un bloque. Empieza pidiendo los nuevos datos con `CapturaBloque`, luego hace un recorrido para encontrar el bloque actual y desconectarlo enlazando el anterior con el siguiente usando `reescribeBloque`, y después guarda y reubica el nuevo bloque actualizado utilizando `escribeBloque` e `InsertaBloque`.

Baja Bloque

Prototipo: void CDiccionario::bajaBloque();

Esta función empieza llamando a `CapturaBloque` para saber qué clave buscar, para después hacer un recorrido donde hace uso de `while` y `comparaBloques` para encontrar dónde está, usa una variable anterior para poder hacer la desconexión del bloque, no borra nada dentro del archivo solo desconecta y conecta con su inmediato siguiente, actualizando el archivo con `reescribeBloque`.

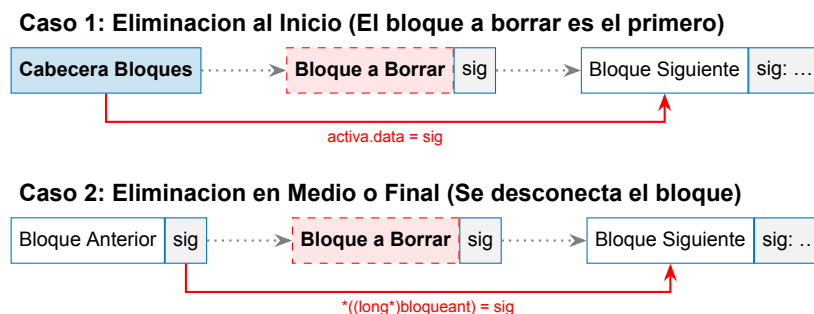


Diagrama 6: Representación gráfica de los 2 casos de baja lógica para la lista de Bloques.

Pide Clave Bloque

Prototipo: `void *CDiccionario::pideClaveBloque();`

Esta función se encarga de crear un bloque temporal exclusivo para la búsqueda, primero pide memoria con `malloc`, después imprime en pantalla el nombre del atributo que es la clave primaria apoyándose de `arrAtributos`, y con un `switch` lee desde el teclado el valor dependiendo de su tipo para guardarlo en el desplazamiento correcto, regresando este bloque para ser usado por funciones de búsqueda o baja.

Elimina Bloque

Prototipo: `void CDiccionario::eliminaBloque(void* bloque);`

Esta función realiza la baja lógica de un registro de datos, recibe el bloque con la clave a buscar y hace un recorrido con un ciclo `while` comparando con `comparaBloques`, cuando encuentra el registro deseado hace la desconexión enlazando el bloque anterior con el siguiente y guardando los cambios con `reescribeBloque`, actualizando la cabecera si es el primer elemento.

6 Diagrama General

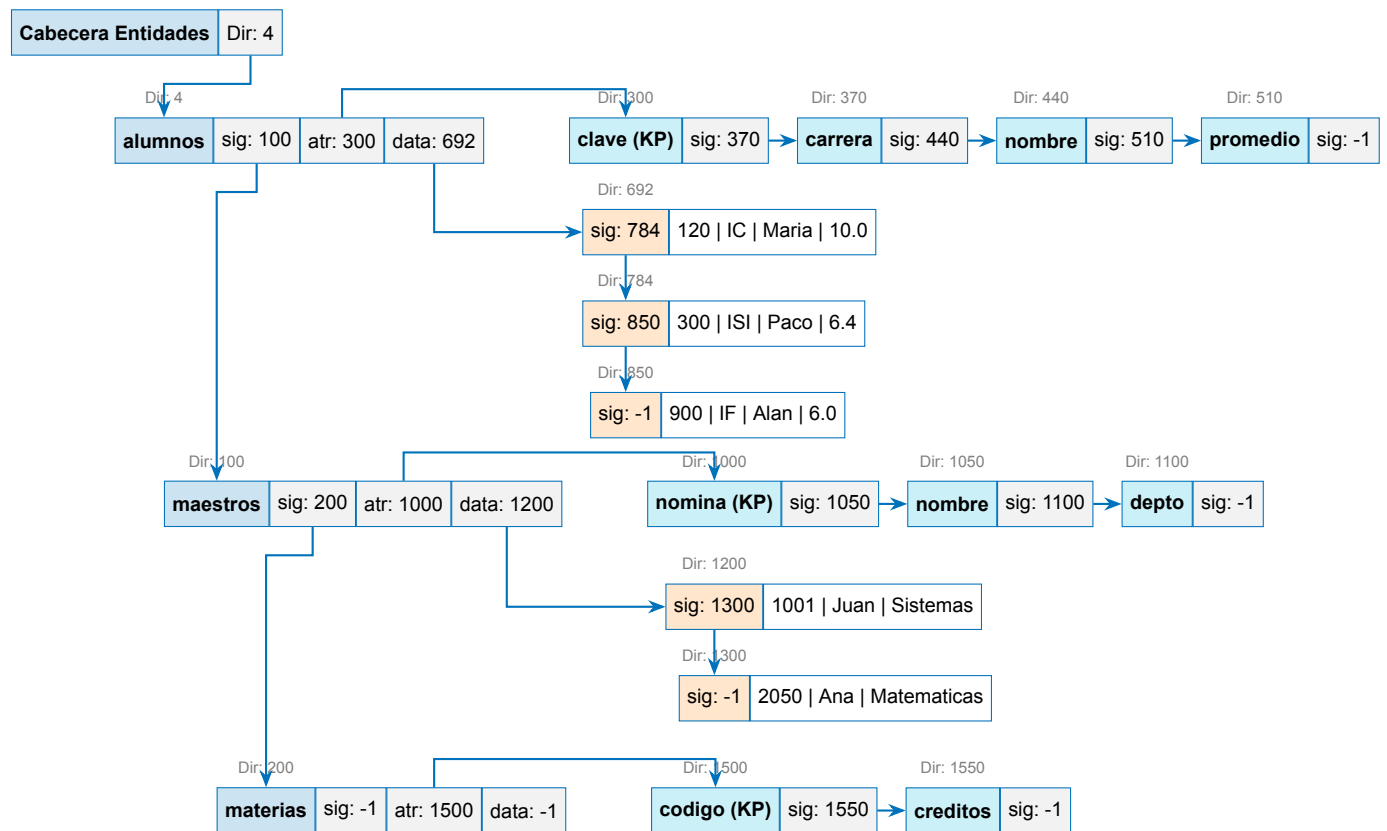


Diagrama 7: Arquitectura del diccionario mostrando la lista principal de Entidades y cómo cada registro conecta con sus respectivas listas de Atributos y Bloques de datos.

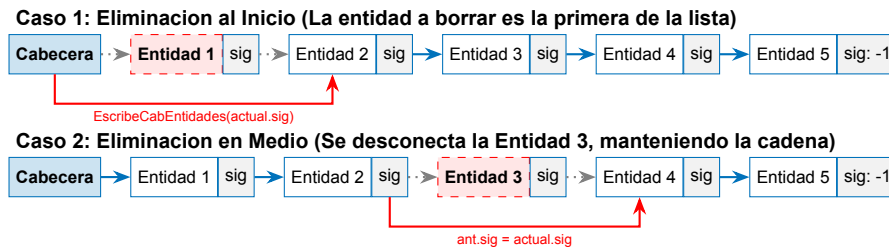
7 Archivo

Cabecera Entidad

Después de que el archivo es creado con la función `void CDiccionario::Nuevo();` y que el usuario agrega la primera entidad por medio de las funciones `altaEntidad` e `insertaEntidad`, el programa al detectar que está vacío va y crea en el primer espacio, podríamos decir en la dirección 0, la cabecera de un tamaño específico de `sizeof(long)`, esta cabecera siempre se almacena al inicio para que apenas se abra el archivo tener acceso directo a la lista.

Eliminación Lógica (Baja)

Cuando el usuario decide borrar un registro usando las funciones de baja, el archivo no reduce su tamaño físico ni borra la información, simplemente hace una desconexión lógica de los apuntadores enlazando el nodo anterior con el siguiente, esto hace que el dato se vuelva inaccesible y quede "puenteado" dentro del documento.



Funciones de Archivo Binario

Para lograr todo este manejo de datos el sistema se apoya de las siguientes funciones nativas de C++:

- **fopen:** Se encarga de abrir el archivo físico en modos binarios como `rb` (lectura) o `wb+` (escritura).
- **fclose:** Cierra el documento para liberar la memoria y guardar los cambios de manera segura.
- **fseek:** Es la función encargada de mover el puntero o cursor a una dirección exacta dentro del archivo.
- **ftell:** Regresa la posición o dirección física exacta en la que se encuentra el cursor en ese momento.
- **fwrite:** Se encarga de escribir o guardar los bloques de datos directamente en el disco.
- **fread:** Hace la operación inversa, leyendo un bloque de bytes desde el archivo para pasarlo a la memoria.

8 Simulaciones de Escritorio: Entidad

1. Simulación: CapturaEntidad

Entrada: Nombre tecleado por el usuario.

Salida: struct Entidad con listas vacías.

Caso	Entrada (cin)	Salida en Memoria (struct)
1	"Materias"	{nombre: "Materias", atr: -1, data: -1, sig: -1}
2	"Alumnos"	{nombre: "Alumnos", atr: -1, data: -1, sig: -1}
3	"Maestros"	{nombre: "Maestros", atr: -1, data: -1, sig: -1}

2. Simulación: escribeEntidad

Entrada: struct Entidad.

Salida: Dirección física asignada (ftell).

Caso (Estado Archivo)	Entidad a escribir	Dir Asignada	Acción en Disco
1. Archivo casi vacío	"Materias"	4	fwrite de 54 bytes en dir 4 (tras cabecera)
2. Archivo con 1 reg.	"Alumnos"	58	fwrite de 54 bytes en dir 58
3. Archivo con 2 reg.	"Maestros"	112	fwrite de 54 bytes en dir 112

3. Simulación: insertaEntidad (Conexión Lógica)

Entrada: Entidad y su dirección física.

Salida: Punteros actualizados en el archivo.

Caso Evaluado	Entra (Dir)	Evaluación strcmp	Resultado Variable
Lista Vacía	Materias (4)	cab == -1	cabecera = 4 Materias.sig = -1
Va al Inicio	Alumnos (58)	Alumnos < Materias	Alumnos.sig = 4 cabecera = 58
Va en Medio/Fin	Maestros (112)	Maestros > Alumnos	Alumnos.sig = 112 Maestros.sig = 4

4. Simulación: ConsultaEntidades

Entrada: cabecera = 58.

Salida: Impresión en pantalla.

Iteración (cab)	leeEntidad(cab)	Pantalla (Nombre)	Siguiente cab (actual.sig)
1 (cab = 58)	Lee dir 58	Alumnos	112
2 (cab = 112)	Lee dir 112	Maestros	4
3 (cab = 4)	Lee dir 4	Materias	-1 (Termina while)

9 Simulaciones de Escritorio: Atributos

1. Simulación: CapturaAt

Entrada: Datos capturados por el usuario (nombre, tipo, longitud, etc.).

Salida: struct Atributo listo para ser escrito.

Caso	Entrada (cin)	Salida en Memoria (struct)
1	"clave", int(2)	{nombre: "clave", tipo: 2, tam: 4, iskp: 'S', sig: -1}
2	"carrera", char(1)	{nombre: "carrera", tipo: 1, tam: 30, iskp: 'N', sig: -1}
3	"nombre", char(1)	{nombre: "nombre", tipo: 1, tam: 50, iskp: 'N', sig: -1}

2. Simulación: EscribeAtributo

Entrada: struct Atributo.

Salida: Dirección física asignada por ftell.

Llamada	Atributo a escribir	Dir Asignada	Acción en Disco
1ra Inserción	"clave"	300	fwrite de sizeof(Atributo) en dir 300
2da Inserción	"carrera"	374	fwrite de sizeof(Atributo) en dir 374
3ra Inserción	"nombre"	448	fwrite de sizeof(Atributo) en dir 448

3. Simulación: insertaAtributo (Conexión Lógica)

Entrada: Atributo nuevo y su dirección física.

Salida: Punteros lógicos (sig) actualizados.

Caso Evaluado	Entra (Dir)	Evaluación strcmp	Resultado Variable
1. Lista Vacía	clave (300)	activa.atr == -1	activa.atr = 300 clave.sig = -1
2. Va al Inicio	carrera (374)	"carrera" < "clave"	carrera.sig = 300 activa.atr = 374
3. Va al Final	nombre (448)	"nombre" > "clave"	clave.sig = 448 nombre.sig = -1

4. Simulación: consultarAtributo

Qué hace: Recorre la lista de atributos saltando por las direcciones en el archivo e imprimiendo.

Entrada: `activa.atr = 374` (Inicio alfabético).

Salida: Impresión en consola de los atributos.

1 (cab = 374)	Lee dir 374	carrera	300
2 (cab = 300)	Lee dir 300	clave (KP)	448
3 (cab = 448)	Lee dir 448	nombre	-1 (Termina while)

10 Simulaciones de Escritorio: Bloques

1. Simulación: CapturaBloque

Entrada: Datos capturados por el usuario según los tipos del arreglo de atributos.

Salida: Puntero `void*` con la información estructurada.

Caso	Entrada (cin)	Salida en Memoria
1	120, "IC", "Maria", 10.0	[sig:-1 120 "IC" "Maria" 10.0]
2	300, "ISI", "Paco", 6.4	[sig:-1 300 "ISI" "Paco" 6.4]
3	900, "IF", "Alan", 6.0	[sig:-1 900 "IF" "Alan" 6.0]

2. Simulación: escribeBloque

Entrada: `void*` bloque y `tambloque`.

Salida: Dirección física asignada por `ftell`.

Llamada	Bloque a escribir	Dir Asignada	Acción en Disco
1ra Inserción	Datos de "Maria"	692	fwrite de <code>tambloque</code> en dir 692
2da Inserción	Datos de "Paco"	784	fwrite de <code>tambloque</code> en dir 784
3ra Inserción	Datos de "Alan"	850	fwrite de <code>tambloque</code> en dir 850

3. Simulación: InsertaBloque (Conexión Lógica)

Entrada: Bloque nuevo y su dirección física.

Salida: Punteros lógicos (`sig`) actualizados en el archivo.

Caso Evaluado	Entra (Dir)	Evaluación <code>comparaBloques</code>	Resultado Variable
1. Lista Vacía	Maria (692)	<code>activa.data == -1</code>	<code>activa.data = 692</code> <code>Maria.sig = -1</code>
2. Va al Final	Paco (784)	<code>300 > 120</code> (Avanza)	<code>Maria.sig = 784</code> <code>Paco.sig = -1</code>
3. Va al Final	Alan (850)	<code>900 > 300</code> (Avanza)	<code>Paco.sig = 850</code> <code>Alan.sig = -1</code>

4. Simulación: consultaBloques

Entrada: activa.data = 692.

Salida: Impresión en consola de todos los registros de la entidad.

Iteración (cab)	LeeBloque(cab)	Pantalla (KP, Car, Nom, Prom)	Siguiente cab
1 (cab = 692)	Lee dir 692	120 IC Maria 10.0	784
2 (cab = 784)	Lee dir 784	300 ISI Paco 6.4	850
3 (cab = 850)	Lee dir 850	900 IF Alan 6.0	-1 (Fin)

11 Clave Primaria

¿Qué es una Clave Primaria?

Una clave primaria en base de datos es aquella que se encarga de la identificación única de ciertos registros, como puede ser el ID de un juego o la clave de la INE; estos valores dentro de la base de datos nunca se deben repetir ya que no pueden existir dos entidades o registros iguales, permitiendo su fácil localización e identificación. A su vez, estas claves primarias son fundamentales para relacionarse con otras tablas, convirtiéndose así en llaves foráneas.

Manejo de la Clave Primaria en el Sistema

Para gestionar esta regla de validación dentro de nuestro archivo binario, el sistema se apoya de las siguientes funciones y lógicas:

- **Funciones de Captura:** La función `CapturaAt` es la encargada de preguntarle al usuario si el nuevo atributo será clave primaria, guardando el indicador en la variable `iskp`. Por otro lado, `CapturaBloque` y `pideClaveBloque` son las funciones responsables de pedirle al usuario el valor numérico o de texto real para esa clave.
- **Función que la compara:** La función `comparaBloques` es la encargada de evaluar exclusivamente la clave primaria de dos registros distintos, determinando cuál va antes o después usando `strcmp` o restas aritméticas según su tipo de dato, siendo esencial para poder insertar y buscar en orden.
- **Validación (existeKP) e Índice 0:** Toda esta lógica está en la función `cargaAtributos`. Mientras lee los atributos de la entidad, utiliza un contador `kp` para validar que exista exactamente una sola clave primaria (cumpliendo la función lógica de *existeKP*); si encuentra cero o más de una, manda un error. Además, al encontrarla hace un intercambio de variables en memoria para asegurar que la clave primaria siempre se almacene y fije en el **índice 0** del arreglo `arrAtributos`, garantizando así que `comparaBloques` y demás funciones siempre evalúen el identificador correcto.

12 Conclusiones

Este proyecto fue una representación en crudo de cómo funcionan las bases de datos primitivas en un lenguaje de bajo nivel. Nos ayudó a comprender de mejor manera cómo funciona la memoria estática (*Archivos*) y cómo esta se va cargando dinámicamente dentro de nuestra memoria RAM. Es importante tener conocimiento de esto, ya que dentro del desarrollo real, como por ejemplo en varios juegos indie, muchas veces guardan las partidas dentro de archivos binarios; de igual manera, los motores de bases de datos operan bajo estos mismos principios, aunque de una forma un tanto diferente y optimizada.

Por otro lado, descubrimos que el manejo del tipo de dato `void*` de alguna forma puede usarse para ofuscar información dentro de los archivos, siendo de único conocimiento del creador del sistema o requiriendo herramientas especializadas para poder traducir y estructurar estos datos en crudo.

Durante las varias fases de desarrollo vimos y aprendimos cómo es ir montando un sistema desde cero usando únicamente las bases de un lenguaje de bajo nivel. Todo este conocimiento puede ser aplicado en la creación de software para componentes de bajos recursos o en sistemas basados en redes privadas para lograr una mayor velocidad. Con unas cuantas adiciones, este modelo base se convertiría en un sistema sumamente rápido y en gran parte confiable.

Otro de los aprendizajes importantes fue comprender la diferencia entre el espacio físico y el lógico en un medio de almacenamiento. Implementar la eliminación lógica nos hizo darnos cuenta de que, en los sistemas a bajo nivel, borrar un registro no significa destruir su información en el disco, sino simplemente desenlazar sus punteros para puentearlo. Esto nos dio una perspectiva muy clara de por qué las bases de datos reales necesitan rutinas de mantenimiento (como el compactado) para recuperar el espacio 'muerto' que se va acumulando.

Finalmente, concluimos que el uso de la clave primaria (*Primary Key*) es una parte vital en la creación de bases de datos. Ésta sirve para la comunicación y relación entre tablas dentro de una misma base, y es el único parámetro que no desaparece en la normalización de los datos. Es una pieza fundamental; el hecho de que solo exista una llave por registro ayuda a localizarlos rápidamente y a mantener un orden estricto, logrando que sean de fácil acceso, de fácil modificación y de fácil borrado, tanto en este sistema que acabamos de desarrollar como en aquellos motores profesionales que se usan en la vida real.